МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3

по курсу «Компьютерная графика»

Текстурирование

Объект: правильный октаэдр

Выполнил: А. А. Устинов
Группа: М8О-307БВ-24

Преподаватель: В. Д. Бахарев

Москва, 2026

1. Условие

Цель работы — загрузить изображения из файловой системы, создать из них текстуры и с помощью текстурных координат вершин и сэмплеров наложить их на рисуемые объекты.

Объект — правильный октаэдр из первой лабораторной работы с освещением по модели Блинна-Фонга из второй. Программа работает в реальном времени с использованием Vulkan, GLFW и ImGui. Альбедо поверхности берётся из текстуры вместо сплошного цвета. Изображения хранятся в файлах PNG и декодируются сторонней библиотекой stb_image. У вершин помимо позиции и нормали есть текстурные координаты, которые генерируются процедурно вместе с геометрией.

Выполнено базовое задание:

- текстура накладывается на объекты как альбедо, без дополнительных эффектов;
- свойства всех моделей хранятся в одном динамическом uniform-буфере, а сдвиг до свойств конкретной модели задаётся в команде привязки набора дескрипторов;
- вершинные и индексные буферы находятся в памяти видеокарты (VRAM) и заполняются командами копирования из staging-буфера;

и два дополнительных задания:

1. загружается шесть текстур, для каждой модели текстура выбирается из выпадающего списка; у каждой текстуры свой набор дескрипторов;
2. для одного объекта реализовано нетривиальное сэмплирование, зависящее от времени: искажение текстурных координат, две выборки одной текстуры разными координатами и модуляция цвета. Включается и настраивается эффект через push-константы.

Для наглядности сцена расширена: октаэдр стоит в комнате из пола и трёх стен, рядом с ним находятся ещё два октаэдра меньшего размера, вращающиеся вокруг вертикальной оси. Всего на сцене семь моделей и три сетки.

2. Метод решения

2.1. Текстуры и их загрузка

Текстура — это изображение в памяти видеокарты, из которого шейдер читает цвет по текстурным координатам $(u, v) \in [0, 1]^2$: точка $(0, 0)$ соответствует левому верхнему углу изображения, $(1, 1)$ — правому нижнему. Отдельный пиксель текстуры называется текселем.

В работе используются шесть бесшовных изображений 512×512: кирпичная кладка, деревянные доски, мрамор, лава, контрольная сетка с подписанными клетками и каменная плитка (рис. 1). Изображения построены из процедурного шума и сохранены в PNG; программа о способе их создания ничего не знает и просто читает файлы.

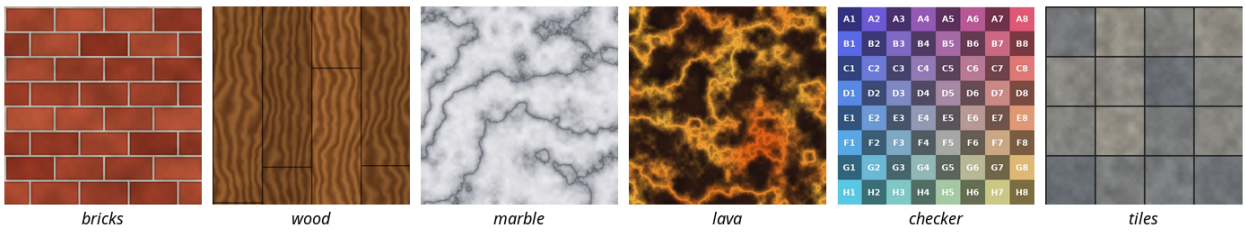


Рисунок 1 – Набор текстур

Файл декодирует функция `stbi_load` библиотеки stb_image. Она поддерживает PNG, JPEG, BMP, TGA и другие форматы и всегда возвращает массив пикселей по четыре байта (R, G, B, A), независимо от числа каналов в файле.

Дальше пиксели нужно переместить в изображение Vulkan. Изображение с оптимальной для видеокарты раскладкой (`VK_IMAGE_TILING_OPTIMAL`) располагается в VRAM, и записать в него напрямую процессор не может. Поэтому пиксели сначала копируются в **staging-буфер** — промежуточный буфер в памяти, доступной процессору, — а затем видеокарта сама переносит их в изображение командой `vkCmdCopyBufferToImage`.

Формат. Изображение создаётся в формате `VK_FORMAT_R8G8B8A8_SRGB`. Цвета в PNG записаны в нелинейном пространстве sRGB, а освещение должно считаться в линейном: складывать и умножать имеет смысл физические величины, а не их гамма-кодированные значения. Для формата `_SRGB` видео-

карта переводит тексели в линейное пространство при чтении в шейдере, а кадр в формате sRGB при записи кодирует обратно.

Раскладки изображения. Изображение Vulkan в каждый момент находится в определённой раскладке (`VkImageLayout`), подходящей для конкретной операции. Переходы выполняются барьерами `vkCmdPipelineBarrier`, которые заодно гарантируют, что следующая операция начнётся только после завершения предыдущей:

Раскладка	Для чего
`UNDEFINED`	начальная, содержимое не определено
`TRANSFER_DST_OPTIMAL`	приёмник копирования
`TRANSFER_SRC_OPTIMAL`	источник копирования
`SHADER_READ_ONLY_OPTIMAL`	чтение из шейдера

Уровни детализации. Если текстура на экране мельче, чем в файле (например, дальняя часть пола), на один пиксель экрана приходится много текселей. Выборка одного из них даёт мерцание и муар. Поэтому для каждой текстуры строится цепочка уменьшенных копий (mip-уровней): каждый следующий уровень вдвое меньше предыдущего, до размера 1×1. Их число

$$L = \left\lfloor \log_2 \max(w, h) \right\rfloor + 1, \tag{1}$$

для 512×512 это 10 уровней. Уровень i получается из уровня $i - 1$ командой `vkCmdBlitImage` с линейной фильтрацией. Перед ней уровень $i - 1$ переводится в раскладку источника, после — в раскладку для чтения шейдером. Видеокарта сама выбирает нужный уровень по тому, насколько быстро меняются текстурные координаты между соседними пикселями экрана. Если формат не поддерживает такое копирование, программа обходится одним уровнем.

Сэмплер описывает, как читается текстура. В работе один сэмплер на все текстуры: линейная фильтрация внутри уровня и между уровнями (трилинейная) и режим адресации `REPEAT`, при котором координаты за пределами $[0, 1]$ берутся по модулю, и текстура повторяется. Повторение нужно полу и стенам, на которых текстура уложена несколько раз, и анимированному эффекту, сдвигающему координаты.

2.2. Текстурные координаты

Текстурные координаты хранятся в каждой вершине вместе с позицией и нормалью и генерируются процедурно.

Пол и стены — прямоугольники, заданные углом **o** и двумя сторонами $\mathbf{e}_u$ и $\mathbf{e}_v$. Вершина с долями (a, b) вдоль сторон, $a, b \in \{0, 1\}$, получает

$$\mathbf{p} = \mathbf{o} + a\,\mathbf{e}_u + b\,\mathbf{e}_v, \qquad (u, v) = \big(a\,n_u,\ (1-b)\,n_v\big), \tag{2}$$

где n_u и n_v — сколько раз текстура укладывается вдоль каждой стороны. Координата v отсчитывается от верхнего края, потому что строки изображения идут сверху вниз; иначе кирпичи на стене оказались бы перевёрнутыми. Пол 12×12 и стены шириной 12 получают по 4 повторения, стены высотой 4,5 — 1,5 повторения, так что одна плитка текстуры везде занимает квадрат 3×3. Координаты больше единицы приводит к $[0, 1]$ режим `REPEAT`.

Октаэдр развёрнут октаэдрической развёрткой, которая отображает поверхность $|x| + |y| + |z| = 1$ на квадрат целиком. Верхняя половина ($y \geq 0$) проецируется на плоскость XZ, а нижняя «раскрывается» наружу:

$$(u', v') = \begin{cases} (x,\ z), & y \geq 0, \\ \big((1 - |z|)\operatorname{sgn} x,\ (1 - |x|)\operatorname{sgn} z\big), & y < 0, \end{cases} \tag{3}$$

после чего координаты переводятся из $[-1, 1]$ в $[0, 1]$:

$$(u, v) = \left(\frac{u' + 1}{2},\ \frac{v' + 1}{2}\right). \tag{4}$$

Четыре верхние грани занимают ромб в центре квадрата, нижние — четыре его угла (рис. 2). Это похоже на конверт: углы квадрата загибаются к центру, и квадрат превращается в октаэдр. Каждая грань отображается на треугольник площадью $1/8$ квадрата аффинно, без искажений, поэтому плотность текселей на всех гранях одинакова и используется вся текстура.

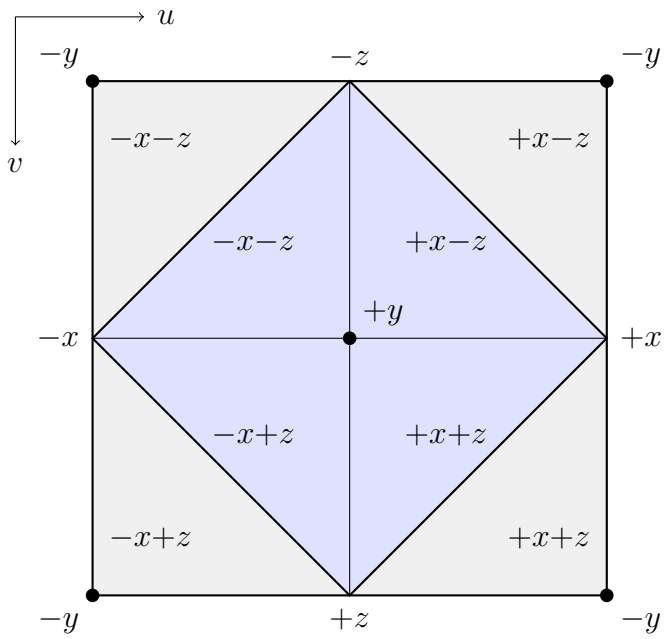


Рисунок 2 – Октаэдрическая развёртка. В ромбе (голубой) — верхние грани, в углах (серые) — нижние; подписаны знаки x и z грани. Вершина $-y$ попадает во все четыре угла квадрата

На рёбрах, соединяющих верхнюю и нижнюю половины, развёртка непрерывна: обе грани отображают ребро на одну и ту же сторону ромба. Шов есть только на четырёх рёбрах, сходящихся в нижней вершине: они лежат на границе квадрата, и соседние грани берут тексели с противоположных его краёв.

Вершины октаэдра по-прежнему дублируются для каждой грани. Причин теперь две: у граней разные нормали (это было и во второй работе), а нижняя вершина получает в каждой из четырёх граней свои текстурные координаты — разные углы квадрата. Принадлежность грани верхней или нижней половине и знаки в формуле (3) определяются по нормали грани, а не по вершине: у вершин на «экваторе» $y = 0$, и по ним одним половину не определить. Для таких вершин обе ветви формулы дают одинаковый результат, поэтому соседние грани сходятся без разрыва.

2.3. Вершины и индексы в VRAM

Во второй работе вершинные буферы находились в памяти, доступной процессору: видеокарта читала их через шину при каждом кадре. Теперь вер-

шины и индексы размещаются в VRAM, а копируются туда так же, как текстуры:

1. создаётся буфер в VRAM с назначением `VERTEX_BUFFER` или `INDEX_BUFFER` и `TRANSFER_DST`; при выделении памяти библиотеке VMA передаётся `VMA_MEMORY_USAGE_AUTO_PREFER_DEVICE` без флагов доступа процессора;
2. данные копируются в staging-буфер в памяти процессора;
3. в буфер команд записывается `vkCmdCopyBuffer` из staging-буфера в буфер VRAM.

Все копирования при запуске — три сетки и шесть текстур — записываются в один буфер команд и отправляются в очередь одним пакетом. В конце записывается барьер, после которого результаты копирования видны этапу чтения вершин и индексов. Программа дожидается выполнения (`vkQueueWaitIdle`), после чего staging-буферы и пул команд уничтожаются.

Индексный буфер позволяет не хранить одинаковые вершины дважды. При построении сетки вершина с теми же позицией, нормалью и текстурными координатами, что уже есть в сетке, не добавляется, а в индексы записывается номер существующей. У прямоугольника два треугольника дают 6 индексов на 4 вершины. У октаэдра повторов нет: 24 индекса на 24 вершины, поскольку у каждой грани свои нормаль и координаты. Рисование выполняется командой `vkCmdDrawIndexed` с 16-битными индексами.

2.4. Динамический uniform-буфер

Во второй работе у каждой модели был собственный uniform-буфер и собственный набор дескрипторов. Теперь свойства всех моделей — матрица преобразования и параметры блика — лежат в одном буфере массивом, а нужный элемент выбирается при привязке набора.

Дескриптор типа `UNIFORM_BUFFER_DYNAMIC` хранит буфер и размер одного элемента, а начальный сдвиг передаётся отдельно в каждой команде `vkCmdBindDescriptorSets`. Шейдер видит одну структуру и не знает, что она — часть массива.

Сдвиг обязан быть кратным пределу видеокарты a — минимальному выравниванию сдвига uniform-буфера (`minUniformBufferOffsetAlignment`).

Структура `ModelData` занимает 80 байт (матрица 4×4 и вектор из четырёх чисел), поэтому шаг между элементами округляется вверх, и модель с номером i находится по сдвигу

$$s = \left\lceil \frac{80}{a} \right\rceil a, \qquad \mathit{offset}_i = i \cdot s. \tag{5}$$

На программном рендерере llvmpipe $a = 16$, и шаг равен 80 байтам; на типичных дискретных видеокартах $a = 64$, и шаг — 128 байт. Каждый кадр процессор записывает свойства всех моделей по своим сдвигам и один раз сбрасывает кеш всего буфера.

Альбедо из свойств модели убрано: цвет поверхности теперь задаёт текстура. В структуре остались матрица, цвет блика и коэффициент блеска.

2.5. Наборы дескрипторов

Внешние данные шейдеров разделены на два набора.

Набор 0 — данные сцены, без изменений со второй работы: uniform-буфер с матрицей $P \cdot V$, положением камеры, рассеянным светом и направленным источником (привязка 0) и storage-буферы с массивами точечных источников и прожекторов (привязки 1 и 2).

Набор 1 — модель и материал:

- привязка 0 — динамический uniform-буфер со свойствами моделей;
- привязка 1 — кортеж изображение-сэмплер (`COMBINED_IMAGE_SAMPLER`) с текстурой альбедо.

В базовом задании все модели используют один такой набор: одна текстура и один буфер, а модели различаются только сдвигом. В дополнительном задании 1 таких наборов шесть, по одному на текстуру. Все они ссылаются на один и тот же динамический буфер, а различаются изображением. Чтобы сменить текстуру модели, достаточно привязать при её отрисовке другой набор: пересоздавать или обновлять дескрипторы не нужно, поэтому текстура меняется в интерфейсе мгновенно.

Порядок команд при отрисовке кадра:

1. привязать конвейер и набор сцены (один раз за кадр);
2. для каждой модели i:

- привязать набор её текстуры с динамическим сдвигом offset_i;
- записать push-константы эффекта;
- привязать вершинный и индексный буферы её сетки и нарисовать.

Три октаэдра используют одну сетку, а стены — другую: модели различаются только матрицей в динамическом буфере. Сетка хранится в VRAM один раз.

2.6. Нетривиальное сэмплирование

Для дополнительного задания 2 во фрагментном шейдере реализован эффект «текущей» поверхности. Обозначим $T(\mathbf{x})$ — выборку из текстуры модели по координатам $\mathbf{x}$, $\mathbf{x}_0 = (u, v)$ — текстурные координаты пикселя, t — время, умноженное на скорость эффекта, d — сила искажения.

1. Искажение координат. К координатам прибавляются две бегущие синусоидальные волны:

$$\mathbf{w} = d \cdot \big(\sin(6\pi v + 2t),\ \ \sin(6\pi u + 1{,}6\,t)\big). \tag{6}$$

Смещение по u зависит от v, и наоборот, поэтому прямые линии текстуры изгибаются волнами, а со временем волны бегут по поверхности.

2. Две выборки одной текстуры разными координатами. Первый слой читается по искажённым координатам, равномерно сдвигающимся по диагонали; второй — по координатам, увеличенным вдвое, искажённым в противоположную сторону и медленно вращающимся вокруг центра развёртки $\mathbf{c} = (0{,}5;\ 0{,}5)$:

$$\mathbf{a} = T\big(\mathbf{x}_0 + \mathbf{w} + (0{,}05;\ 0{,}03)\,t\big), \qquad \mathbf{b} = T\big(2\,R(0{,}2\,t)(\mathbf{x}_0 - \mathbf{c}) + \mathbf{c} - \mathbf{w}\big), \tag{7}$$

где $R(\alpha)$ — матрица поворота на угол α. Слои объединяются покомпонентным максимумом $\max(\mathbf{a}, \mathbf{b})$: светлые участки обоих слоёв остаются видны, и рисунки двигаются друг сквозь друга. Режим `REPEAT` делает сдвиг и увеличение координат бесшовными.

3. Модуляция цвета. Результат умножается на оттенок, плавно обходящий цветовой круг, и на яркость, пульсирующую волной вдоль диагонали

развёртки:

$$\mathbf{k}_d = 1{,}4\ \max(\mathbf{a}, \mathbf{b}) \odot \boldsymbol{\tau} \cdot \rho, \tag{8}$$

$$\boldsymbol{\tau} = 0{,}6 + 0{,}4\cos\big(2\pi(0{,}08\,t + (0;\ 0{,}33;\ 0{,}67))\big), \tag{9}$$

$$\rho = 0{,}85 + 0{,}15\sin\big(3t - 8(u+v)\big). \tag{10}$$

Сдвиг фаз каналов на треть периода даёт поочерёдное усиление красного, зелёного и синего, то есть поворот оттенка. Полученное $\mathbf{k}_d$ дальше используется в модели Блинна-Фонга как обычное альбедо, поэтому объект с эффектом освещается так же, как остальные.

Push-константы. Включение эффекта и его параметры передаются не в uniform-буфере моделей, а в push-константах — небольшом блоке данных (по стандарту гарантировано не меньше 128 байт), который записывается прямо в буфер команд командой `vkCmdPushConstants`:

```
layout(push_constant) uniform Effect {
    uint enabled;      // 0 - обычная выборка, 1 - анимированная
    float time;        // время в секундах
    float speed;       // множитель скорости
    float distortion;  // сила искажения
} effect;
```

Эти 16 байт записываются перед каждой отрисовкой, для моделей без эффекта `enabled = 0`. Для них не нужны ни буфер, ни дескриптор, ни выравнивание, а структура свойств модели не раздувается полями, нужными одному объекту. Диапазон push-констант объявлен в раскладке конвейера только для фрагментного шейдера.

2.7. Используемые объекты Vulkan

- `VkImage` и `VkImageView` — по одному на текстуру: изображение в VRAM со всеми mip-уровнями и вид на него для шейдера;
- `VkSampler` — один сэмплер на все текстуры;
- `VkBuffer` в VRAM — вершинный и индексный буферы каждой из трёх сеток (октаэдр, пол, стена);

- `VkBuffer` в памяти процессора — uniform-буфер сцены, storage-буферы источников, динамический uniform-буфер моделей и временные staging-буферы;
- `VkCommandPool` и `VkCommandBuffer` — для копирования данных в VRAM при запуске; уничтожаются сразу после копирования;
- `VkDescriptorSetLayout` — две раскладки: сцены и материала;
- `VkDescriptorPool` и `VkDescriptorSet` — один набор сцены и шесть наборов материалов. Все они заполняются один раз при запуске, поскольку ни буферы, ни изображения не пересоздаются;
- `VkPipelineLayout` — два набора дескрипторов и диапазон push-констант размером 16 байт;
- `VkPipeline` — графический конвейер. В формат вершины добавлен третий атрибут — текстурные координаты (`R32G32_SFLOAT`); остальные настройки совпадают со второй работой.

2.8. Интерфейс

Раздел камеры и разделы источников света сохранены со второй работы. В новом разделе моделей у каждой из семи моделей есть выпадающий список текстур (дополнительное задание 1), положение, цвет блика и коэффициент блеска, у октаэдров — масштаб и скорость вращения. Флажок «Animated sampling» включает анимированное сэмплирование, а ползунки под ним задают скорость эффекта и силу искажения (дополнительное задание 2). По умолчанию эффект включён только у центрального октаэдра.

Ниже приведён фрагментный шейдер с выборкой из текстуры и анимированным эффектом.

```
#version 450

layout(location = 0) in vec3 world_position;
layout(location = 1) in vec3 world_normal;
layout(location = 2) in vec2 uv;

```

```
layout(location = 0) out vec4 out_color;

layout(set = 0, binding = 0) uniform SceneData {
    mat4 view_projection;
    vec4 camera_position;
    vec4 ambient;
    vec4 light_direction;
    vec4 light_color;
    uint point_light_count;
    uint spot_light_count;
} scene;

struct PointLight {
    vec4 position;          // xyz
    vec4 color;             // rgb — цвет, a — интенсивность
};

struct SpotLight {
    vec4 position;          // xyz
    vec4 direction;         // xyz — куда светит
    vec4 color;             // rgb — цвет, a — интенсивность
    vec4 cutoff;            // x — косинус внутреннего угла, y — внешнего
};

layout(std430, set = 0, binding = 1) readonly buffer PointLights {
    PointLight point_lights[];
};

layout(std430, set = 0, binding = 2) readonly buffer SpotLights {
    SpotLight spot_lights[];
};

layout(set = 1, binding = 0) uniform ModelData {
    mat4 transform;
    vec4 specular;
} object;

// Набор 1, привязка 1 — текстура альбедо вместе с сэмплером.
// У каждой текстуры свой набор (доп. задание 1)
layout(set = 1, binding = 1) uniform sampler2D albedo_texture;

// Доп. задание 2: параметры нетривиального сэмплирования приходят в
// push-константах, а не в uniform-буфере модели
layout(push_constant) uniform Effect {
    uint enabled;           // 0 — обычная выборка, 1 — анимированная
    float time;             // время в секундах
```

```
    float speed;            // множитель скорости анимации
    float distortion;       // сила искажения текстурных координат
} effect;

const float TAU = 6.28318530718;

// Поворачивает точку на плоскости на угол a вокруг начала координат.
vec2 rotate(vec2 p, float a) {
    float s = sin(a);
    float c = cos(a);
    return vec2(c * p.x - s * p.y, s * p.x + c * p.y);
}

// Доп. задание 2: нетривиальное сэмплирование, зависящее от времени.
vec3 animatedAlbedo(vec2 coords) {
    float t = effect.time * effect.speed;

    // 1. Искажение координат: две бегущие синусоидальные волны
    vec2 wave = effect.distortion * vec2(
        sin(coords.y * 3.0 * TAU + 2.0 * t),
        sin(coords.x * 3.0 * TAU + 1.6 * t));

    // 2. Две выборки одной и той же текстуры разными координатами:
    // первый слой сдвигается по диагонали, второй увеличен вдвое
    // и медленно вращается вокруг центра развёртки
    vec2 uv_a = coords + wave + vec2(0.05, 0.03) * t;
    vec2 uv_b = rotate(coords - 0.5, 0.2 * t) * 2.0 + 0.5 - wave;

    vec3 a = texture(albedo_texture, uv_a).rgb;
    vec3 b = texture(albedo_texture, uv_b).rgb;

    // Слои смешиваются: яркие участки обоих слоёв остаются видны
    vec3 layers = max(a, b);

    // 3. Модуляция цвета: оттенок плавно обходит цветовой круг,
    // а яркость пульсирует волной, бегущей по поверхности
    vec3 tint = 0.6 + 0.4 * cos(TAU * (0.08 * t + vec3(0.0, 0.33, 0.67)));
    float pulse = 0.85 + 0.15 * sin(3.0 * t - 8.0 * (coords.x + coords.y));

    return layers * tint * pulse * 1.4;
}

// Цвет поверхности в точке: из текстуры, обычной или анимированной выборкой.
vec3 albedo() {
    if (effect.enabled != 0) {
        return animatedAlbedo(uv);
```

```
    }

    return texture(albedo_texture, uv).rgb;
}

// Модель Блинна-Фонга для одного источника.
// N — нормаль, V — направление на камеру, L — направление на источник,
// kd — альбедо из текстуры, radiance — свет, пришедший от источника.
vec3 blinnPhong(vec3 N, vec3 V, vec3 L, vec3 kd, vec3 radiance) {
    float n_dot_l = max(dot(N, L), 0.0);
    if (n_dot_l == 0.0) {
        return vec3(0.0);
    }

    vec3 H = normalize(L + V);
    float n_dot_h = max(dot(N, H), 0.0);

    vec3 diffuse = kd * n_dot_l;
    vec3 specular = object.specular.rgb * pow(n_dot_h, object.specular.a);

    return (diffuse + specular) * radiance;
}

// Выполняется для каждого пикселя: берёт альбедо из текстуры и складывает
// вклад всех источников света.
void main() {
    vec3 N = normalize(world_normal);
    vec3 V = normalize(scene.camera_position.xyz - world_position);
    vec3 kd = albedo();

    // Рассеянный свет
    vec3 color = scene.ambient.rgb * kd;

    // Направленный источник
    vec3 sun_direction = normalize(-scene.light_direction.xyz);
    color += blinnPhong(N, V, sun_direction, kd, scene.light_color.rgb * scene.light_color.a);

    // Точечные источники, затухание по закону обратных квадратов
    for (uint i = 0; i < scene.point_light_count; ++i) {
        PointLight light = point_lights[i];

        vec3 to_light = light.position.xyz - world_position;
        float distance_squared = max(dot(to_light, to_light), 0.0001);
        vec3 L = to_light / sqrt(distance_squared);

        vec3 radiance = light.color.rgb * light.color.a / distance_squared;
```

```
        color += blinnPhong(N, V, L, kd, radiance);
    }

    // Прожекторы: затухание и мягкий край пятна
    for (uint i = 0; i < scene.spot_light_count; ++i) {
        SpotLight light = spot_lights[i];

        vec3 to_light = light.position.xyz - world_position;
        float distance_squared = max(dot(to_light, to_light), 0.0001);
        vec3 L = to_light / sqrt(distance_squared);

        float cos_angle = dot(-L, normalize(light.direction.xyz));
        float cos_inner = light.cutoff.x;
        float cos_outer = light.cutoff.y;
        float edge = clamp((cos_angle - cos_outer) / max(cos_inner - cos_outer, 0.0001),
                           0.0, 1.0);

        vec3 radiance = light.color.rgb * light.color.a * edge / distance_squared;
        color += blinnPhong(N, V, L, kd, radiance);
    }

    out_color = vec4(color, 1.0);
}
```

Листинг 1: Фрагментный шейдер: выборка из текстуры, анимированное сэмплирование и модель Блинна-Фонга

3. Результаты

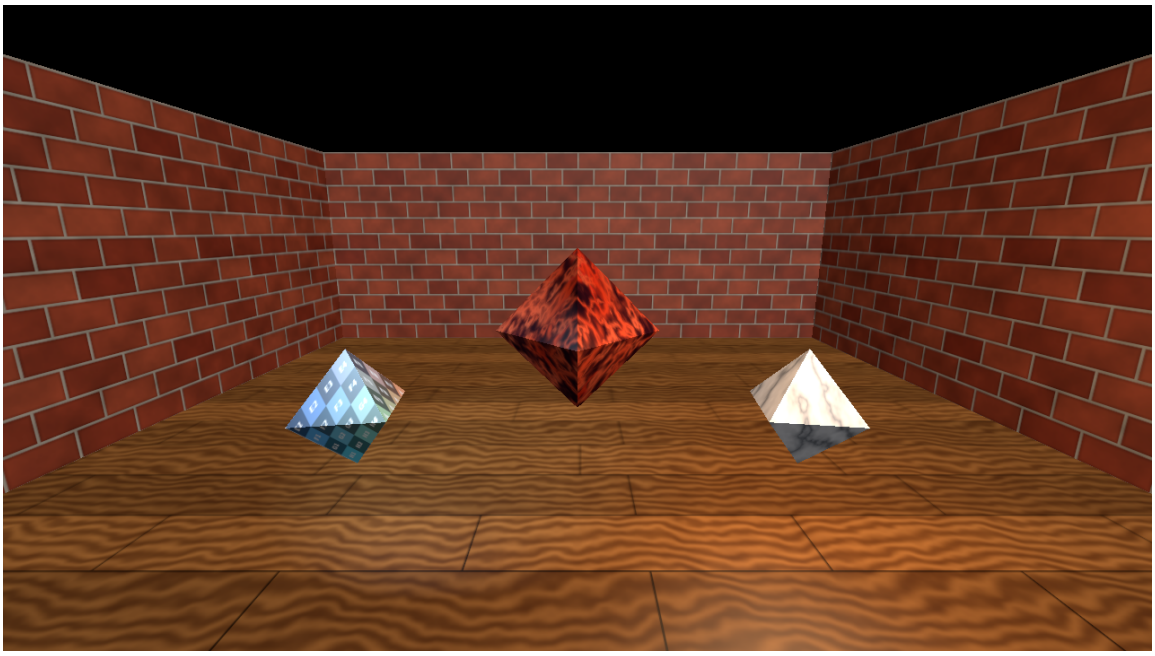


Рисунок 3 – Сцена при запуске. Пол — деревянные доски, стены — кирпич, текстура на них повторяется. Центральный октаэдр — лава с анимированным сэмплированием, слева — контрольная сетка, справа — мрамор

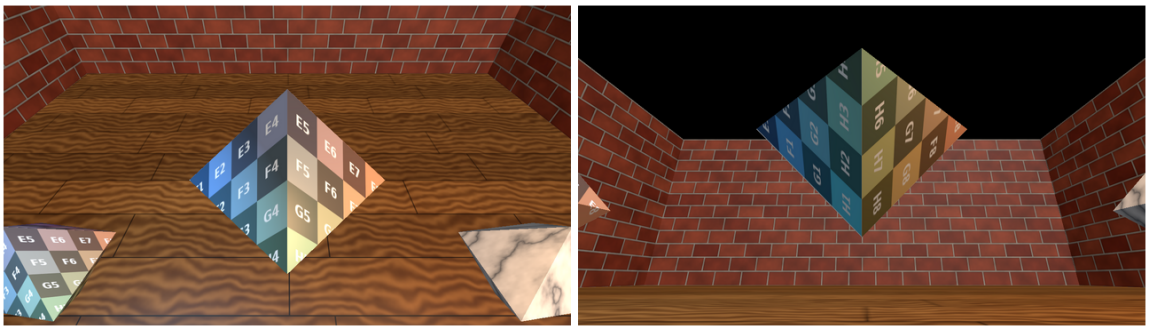


Рисунок 4 – Октаэдрическая развёртка на контрольной сетке. Слева — вид сверху: верхние грани показывают центральный ромб текстуры (в верхней вершине сходятся клетки D4, D5, E4 и E5, спереди видны строки E–H). Справа — вид снизу: нижние грани показывают углы квадрата; надписи на них повёрнуты, но не отражены зеркально, потому что развёртка сохраняет ориентацию граней

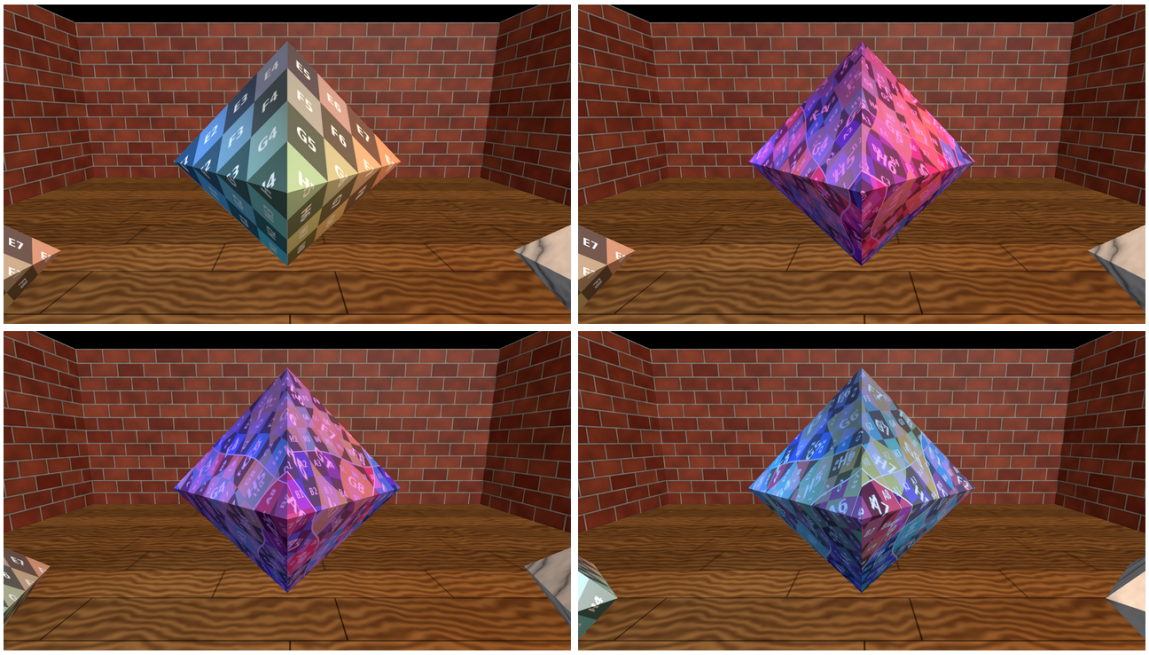


Рисунок 5 – Анимированное сэмплирование на контрольной сетке. Слева сверху — эффект выключен, остальные кадры сняты с интервалом около 1,5 с при включённом эффекте: видны два слоя текстуры разного масштаба, волновое искажение надписей и смена оттенка

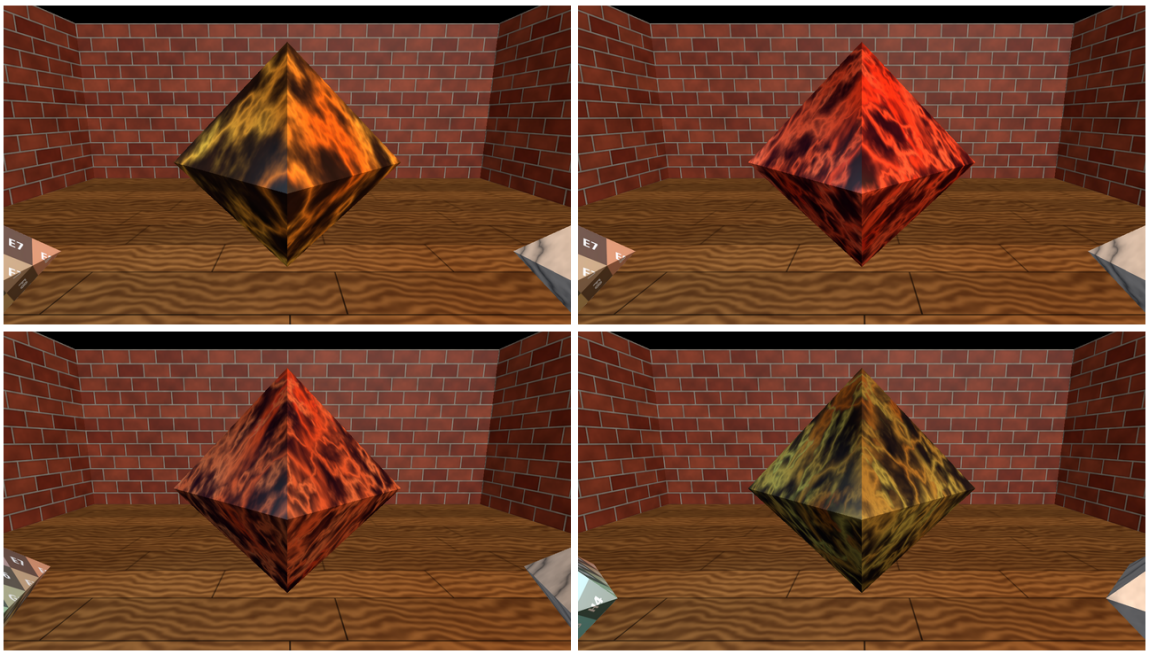


Рисунок 6 – Тот же эффект на текстуре лавы. Слева сверху — без эффекта; на остальных кадрах трещины двух слоёв перемещаются друг относительно друга, а цвет переходит от красного к жёлто-зелёному

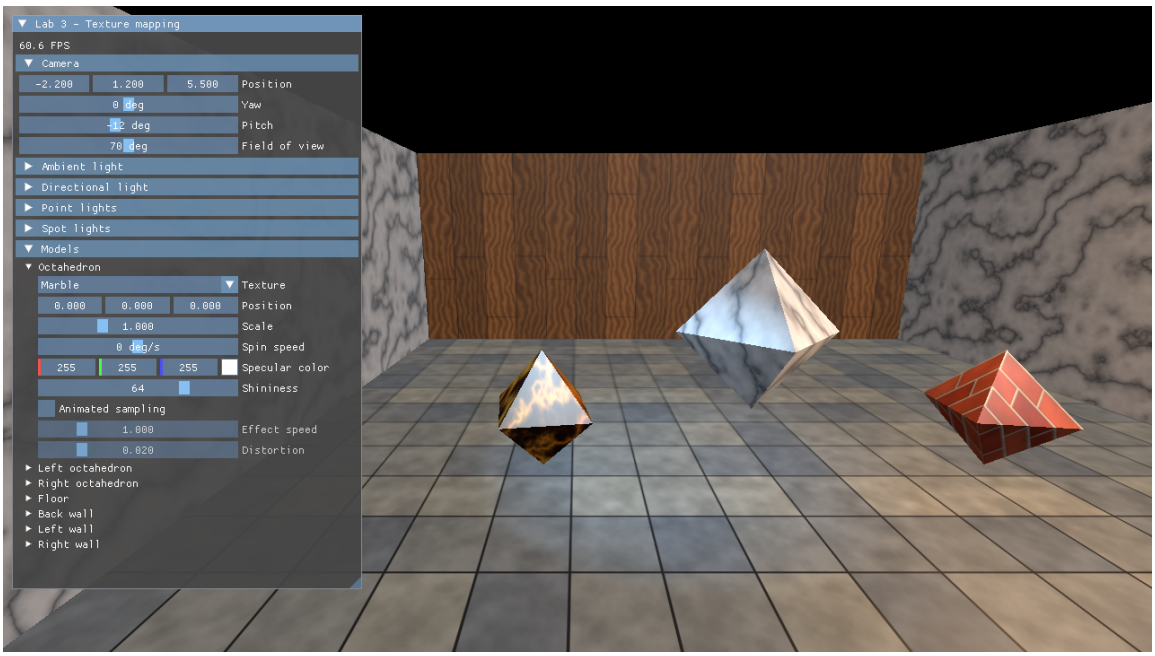


Рисунок 7 – Смена текстур через интерфейс: пол — каменная плитка, задняя стена — доски, боковые — мрамор; центральный октаэдр — мрамор без эффекта, правый — кирпич. Голубое пятно на полу — прожектор

4. Выводы

В ходе работы я научился загружать изображения из файлов, создавать из них текстуры Vulkan и накладывать их на объекты с помощью текстурных координат и сэмплера, а также переносить данные в память видеокарты через staging-буферы.

Главное, что я понял: копирование в VRAM — это не вызов `memcpy`, а команда, которую выполняет видеокарта. Её нужно записать в буфер команд, отправить в очередь и дождаться, а для изображений ещё и явно перевести их из одной раскладки в другую барьерами. Зато после этого вершины, индексы и текстуры лежат в быстрой памяти, и каждый кадр видеокарта читает их, не обращаясь к памяти процессора.

Текстурные координаты, как и нормали во второй работе, оказались свойством грани, а не вершины: нижняя вершина октаэдра получает в четырёх гранях четыре разные пары координат. Октаэдрическая развёртка позволила уложить на октаэдр всю текстуру целиком и без искажений, потому что каждая грань отображается на свой треугольник квадрата аффинно.

Динамический uniform-буфер заменил множество отдельных буферов

одним: модели различаются только сдвигом при привязке набора. Наборы дескрипторов на каждую текстуру позволяют сменить текстуру модели одной привязкой другого набора, без обновления дескрипторов. Push-константы удобны для небольших параметров, которые меняются от объекта к объекту, — флага и параметров эффекта: они не требуют ни буфера, ни дескриптора.